

PostgreSQL Encryption — pgcrypto and Data-at-Rest

Table of Contents

1. [Learning Objectives](#)
 2. [Encryption Layers in PostgreSQL](#)
 3. [pgcrypto Extension](#)
 4. [Symmetric Encryption Functions](#)
 5. [Asymmetric Encryption \(PGP\)](#)
 6. [Hashing Functions](#)
 7. [Column-Level Encryption Patterns](#)
 8. [Transparent Data Encryption \(TDE\)](#)
 9. [Key Management](#)
 10. [Performance Impact](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Interview Questions](#)
 14. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Use pgcrypto for column-level encryption
 - Implement password hashing with bcrypt/scrypt
 - Understand symmetric vs asymmetric encryption trade-offs
 - Design an encryption key management strategy
 - Evaluate when to use column-level vs filesystem-level encryption
 - Implement PGP encryption for sensitive data
-

Encryption Layers in PostgreSQL

LAYER 1: Network Encryption

Client →SSL/TLS→ PostgreSQL
(protects data in transit)

LAYER 2: Column-Level Encryption (pgcrypto)

Application encrypts data before storing in columns
Data stored as: encrypted bytea or text
(protects against DB admin reading sensitive columns)

LAYER 3: Tablespace/Filesystem Encryption

OS-level encryption (LUKS, FileVault, EBS encryption)
Protects disk files if storage media is stolen
(TDE - Transparent Data Encryption)

LAYER 4: Backup Encryption

```
pgBackRest, WAL-G encrypt backup files
(protects backups at rest)
```

pgcrypto Extension

```
-- Install pgcrypto
CREATE EXTENSION pgcrypto;

-- Verify installation
SELECT * FROM pg_extension WHERE extname = 'pgcrypto';

-- Available functions overview
\df pgp_*
\df crypt*
\df digest*
\df encrypt*
\df gen_*
```

Symmetric Encryption Functions

encrypt() / decrypt()

```
-- Symmetric encryption using AES-256-CBC
-- Key should be 16, 24, or 32 bytes (128, 192, 256-bit AES)

-- Encrypt a value
SELECT encrypt('sensitive_data'::bytea, 'encryption_key_32bytes!!!!!!!!!!', 'aes');
-- Returns: bytea (binary encrypted data)

-- Decrypt it back
SELECT convert_from(
    decrypt(encrypted_value, 'encryption_key_32bytes!!!!!!!!!!', 'aes'),
    'UTF8'
) AS decrypted_text;

-- Store as hex for text columns
SELECT encode(
    encrypt('Alice Johnson'::bytea, 'my_encryption_key_32bytes_long!!', 'aes'),
    'hex'
) AS encrypted_hex;

-- Decrypt from hex
SELECT convert_from(
    decrypt(
        decode('hex_encoded_encrypted_value', 'hex'),
        'my_encryption_key_32bytes_long!!',
        'aes'
    ),
```

```
'UTF8'
) AS decrypted;
```

Cipher Modes

```
-- Available encryption types in pgcrypto:
-- aes          : AES with CBC mode (default, widely supported)
-- aes-ecb      : AES ECB (NOT recommended - same plaintext = same ciphertext)
-- aes-cbc      : AES CBC (explicit, same as 'aes')

-- 3DES
SELECT encrypt('data'::bytea, 'key-must-be-24-bytes-long!', '3des');
```

```
-- Blowfish
SELECT encrypt('data'::bytea, 'any-length-key', 'bf');
```

```
-- Example with IV (Initialization Vector) for better security:
SELECT encrypt_iv(
    'sensitive data'::bytea,
    'encryption_key_32bytes!!!!!!!!!!',
    gen_random_bytes(16),    -- Random IV
    'aes'
);
-- Note: you must store the IV alongside the ciphertext to decrypt later!
```

Asymmetric Encryption (PGP)

PGP (Pretty Good Privacy) uses public/private key pairs. Encrypt with public key, decrypt with private key.

PGP Symmetric (Passphrase-based)

```
-- PGP symmetric encryption (uses passphrase)
-- More secure than raw symmetric: uses proper key derivation

-- Encrypt
SELECT pgp_sym_encrypt(
    'Sensitive PII data',      -- plaintext
    'my_strong_passphrase',    -- passphrase
    'cipher-algo=aes256, compress-algo=1' -- options
) AS pgp_ciphertext;
```

```
-- Decrypt
SELECT pgp_sym_decrypt(
    pgp_ciphertext_column,
    'my_strong_passphrase'
) AS plaintext
FROM table_with_encrypted_data
WHERE id = 1;
```

```
-- With metadata inspection
SELECT pgp_sym_decrypt_bytea(ciphertext, passphrase) AS raw_bytes;
```

PGP Asymmetric (Public/Private Key)

```
# Generate PGP key pair for database encryption
gpg --batch --gen-key << EOF
%no-protection
Key-Type: RSA
Key-Length: 4096
Subkey-Type: RSA
Subkey-Length: 4096
Name-Real: DB Encryption Key
Name-Email: db@example.com
Expire-Date: 0
%commit
EOF

# Export keys
gpg --armor --export db@example.com > public.key
gpg --armor --export-secret-keys db@example.com > private.key
```

```
-- Create a table to store the keys (or use Vault)
CREATE TABLE encryption_keys (
    key_id TEXT PRIMARY KEY,
    public_key TEXT NOT NULL,
    -- private_key is stored securely outside PostgreSQL
    created_at TIMESTAMPTZ DEFAULT now()
);

INSERT INTO encryption_keys (key_id, public_key)
VALUES ('db_key_2024', pgp_public_key); -- Insert from application

-- Encrypt using public key (any app can encrypt)
SELECT pgp_pub_encrypt(
    'SSN: 123-45-6789',
    dearmor(public_key) -- dearmor converts ASCII armor to binary
) AS encrypted_ssn
FROM encryption_keys
WHERE key_id = 'db_key_2024';

-- Decrypt requires the private key (only secure app/admin)
SELECT pgp_pub_decrypt(
    encrypted_column,
    dearmor('-----BEGIN PGP PRIVATE KEY BLOCK-----
...
-----END PGP PRIVATE KEY BLOCK-----')
) AS decrypted_ssn
```

```
FROM sensitive_table
WHERE id = 1;
```

Hashing Functions

```
-- Cryptographic hashing (one-way, not encryption!)
-- Use for: password storage, data integrity, deduplication

-- SHA-256
SELECT encode(digest('password123', 'sha256'), 'hex');

-- SHA-512
SELECT encode(digest('password123', 'sha512'), 'hex');

-- MD5 (weak, don't use for passwords)
SELECT md5('data');
```

```
-- Multiple algorithms
SELECT encode(digest('data', algorithm), 'hex')
FROM (VALUES ('sha256'), ('sha512'), ('sha1')) AS t(algorithm);

-- HMAC (keyed hash - for authentication)
SELECT encode(hmac('message', 'secret_key', 'sha256'), 'hex');
SELECT encode(hmac('data', 'api_secret', 'sha512'), 'hex');
```

Password Hashing with bcrypt/script

```
-- bcrypt (recommended for passwords)
-- The 8 = work factor (rounds = 2^8 = 256, typical: 8-12)
-- Work factor makes brute-force attacks expensive

-- Hash a password
SELECT crypt('user_password', gen_salt('bf', 10));
-- Returns: $2a$10$<salt+hash> (62 chars)

-- Verify a password
SELECT crypt('user_password', stored_hash) = stored_hash AS password_matches;
-- Or:
SELECT crypt(input_password, hash_from_db) = hash_from_db AS matches
FROM users WHERE username = 'alice';

-- MD5 (NOT for passwords - just for data integrity)
SELECT gen_salt('md5'); -- Only 8 bit salt - too weak

-- SHA-256 based crypt
SELECT crypt('password', gen_salt('sha256'));

-- Recommended: bcrypt with work factor 10-12
```

```
SELECT crypt('my_secure_password', gen_salt('bf', 12));
-- Returns: $2a$12$... (will take ~300ms to compute - brute-force protection)
```

Column-Level Encryption Patterns

Pattern 1: PII Encryption (SSN, Credit Cards)

```
-- Table with encrypted sensitive columns
CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,           -- Searchable, not encrypted
  email TEXT NOT NULL UNIQUE,   -- Searchable, not encrypted
  ssn_encrypted BYTEA,          -- Encrypted SSN
  credit_card_encrypted BYTEA,  -- Encrypted credit card
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Application encrypts before storing:
-- Using pgp_sym_encrypt:
INSERT INTO customers (name, email, ssn_encrypted, credit_card_encrypted)
VALUES (
  'Alice Johnson',
  'alice@example.com',
  pgp_sym_encrypt('123-45-6789', current_setting('app.encryption_key')),
  pgp_sym_encrypt('4111111111111111', current_setting('app.encryption_key'))
);

-- Application decrypts when needed:
SELECT
  name,
  email,
  pgp_sym_decrypt(ssn_encrypted, current_setting('app.encryption_key')) AS ssn,
  pgp_sym_decrypt(credit_card_encrypted, current_setting('app.encryption_key')) AS
credit_card
FROM customers
WHERE id = 1;
```

Pattern 2: Searchable Encryption (Tokenization)

```
-- You can't directly search encrypted values
-- Solution: store a hash (token) alongside the encrypted value

CREATE TABLE payment_methods (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL,
  card_number_encrypted BYTEA,  -- Full card number, encrypted
  card_last_four TEXT,         -- Last 4 digits, plaintext (for display)
  card_hash TEXT,              -- SHA-256 of full number (for duplicate
detection)
```

```

        card_type TEXT
    );

INSERT INTO payment_methods (user_id, card_number_encrypted, card_last_four,
card_hash, card_type)
VALUES (
    42,
    pgp_sym_encrypt('4111111111111111', 'encryption_key'),
    '1111',
    encode(digest('4111111111111111', 'sha256'), 'hex'),
    'Visa'
);

-- Check if card already exists (without decrypting all rows)
SELECT id FROM payment_methods
WHERE card_hash = encode(digest('4111111111111111', 'sha256'), 'hex');

```

Pattern 3: Audit Log with Encrypted Data

```

CREATE TABLE audit_log (
    id BIGSERIAL PRIMARY KEY,
    user_id INT,
    action TEXT,
    table_name TEXT,
    old_data_encrypted BYTEA, -- Encrypted old row state
    new_data_encrypted BYTEA, -- Encrypted new row state
    metadata JSONB,           -- Non-sensitive metadata (searchable)
    created_at TIMESTAMPTZ DEFAULT now()
);

-- Trigger function to auto-encrypt audit data
CREATE OR REPLACE FUNCTION audit_trigger_fn()
RETURNS TRIGGER AS $$
DECLARE
    enc_key TEXT := current_setting('app.audit_encryption_key', true);
BEGIN
    IF TG_OP IN ('UPDATE', 'DELETE') THEN
        INSERT INTO audit_log(user_id, action, table_name, old_data_encrypted,
metadata)
        VALUES (
            current_setting('app.user_id', true)::INT,
            TG_OP,
            TG_TABLE_NAME,
            pgp_sym_encrypt(row_to_json(OLD)::text, enc_key),
            jsonb_build_object('timestamp', now(), 'session_user', session_user)
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

Transparent Data Encryption (TDE)

PostgreSQL does NOT have native TDE in the open-source version (unlike Oracle/SQL Server). Options:

Option 1: OS-Level Filesystem Encryption

```
# LUKS full-disk encryption on Linux
# Encrypts the entire filesystem including PostgreSQL data directory

# Create encrypted partition
cryptsetup luksFormat /dev/sdb
cryptsetup luksOpen /dev/sdb pg_data_encrypted
mkfs.ext4 /dev/mapper/pg_data_encrypted
mkdir -p /var/lib/postgresql/encrypted
mount /dev/mapper/pg_data_encrypted /var/lib/postgresql/encrypted

# Move PostgreSQL data directory to encrypted partition
sudo systemctl stop postgresql
rsync -av /var/lib/postgresql/16/main/ /var/lib/postgresql/encrypted/main/
sudo -u postgres pg_ctl start -D /var/lib/postgresql/encrypted/main
```

Option 2: EBS Encryption (AWS)

```
# Amazon EBS volumes with encryption at rest
# All data written to the volume is automatically encrypted

# Create encrypted EBS volume
aws ec2 create-volume \
  --size 500 \
  --region us-east-1 \
  --availability-zone us-east-1a \
  --volume-type gp3 \
  --encrypted \
  --kms-key-id alias/my-rds-key

# Mount and use as PostgreSQL data directory
```

Option 3: PostgreSQL TDE (Percona, EDB Enterprise, PG 17 Encrypted Storage)

```
# Percona Distribution for PostgreSQL with TDE
# Encrypts WAL and data files at the PostgreSQL cluster level

# EDB Postgres Advanced Server (enterprise)
# Supports TDE natively with key management integration
```

Key Management


```

-- Application-level key management strategies

-- Strategy 1: Key in application config (simplest, least secure)
-- Connection sets the key:
-- SET app.encryption_key = 'key_from_env_var';
-- Policy: the key comes from APP, not stored in DB

-- Strategy 2: Key in HashiCorp Vault
-- Application fetches key from Vault at startup
-- Key is in memory only, never stored in DB or config files

-- Strategy 3: Key derivation per record
-- Master key + record ID → derived key per record
-- If one record's key is compromised, others are safe

CREATE OR REPLACE FUNCTION derive_record_key(
    master_key BYTEA,
    record_id BIGINT
) RETURNS BYTEA AS $$
    SELECT hmac(record_id::text::bytea, master_key, 'sha256');
$$ LANGUAGE sql IMMUTABLE STRICT;

-- Encrypt with per-record key
INSERT INTO sensitive_records (id, data_encrypted)
VALUES (
    42,
    pgp_sym_encrypt(
        'sensitive data',
        encode(derive_record_key('master_key_bytes'::bytea, 42), 'escape')
    )
);

-- Strategy 4: Envelope encryption
-- Data encrypted with random Data Encryption Key (DEK)
-- DEK encrypted with Key Encryption Key (KEK) from Vault
CREATE TABLE encrypted_documents (
    id SERIAL PRIMARY KEY,
    content_encrypted BYTEA,      -- Encrypted with random DEK
    dek_encrypted BYTEA          -- DEK encrypted with KEK (from Vault)
);

```

Performance Impact

```

-- Benchmark pgcrypto overhead
DO $$
DECLARE
    start_time TIMESTAMPTZ;
    end_time TIMESTAMPTZ;
    i INT;

```

```

BEGIN
    start_time := clock_timestamp();
    FOR i IN 1..10000 LOOP
        PERFORM pgp_sym_encrypt('test data', 'encryption_key_32bytes_long!!');
    END LOOP;
    end_time := clock_timestamp();
    RAISE NOTICE 'pgp_sym_encrypt: % ms for 10000 operations',
        EXTRACT(MILLISECONDS FROM (end_time - start_time));
END;
$$;

-- Typical overhead:
-- pgp_sym_encrypt: ~0.1-0.5ms per operation
-- bcrypt(work=10): ~100ms per hash (by design!)
-- SHA-256: ~0.01ms per hash
-- AES-256 (encrypt): ~0.05ms per operation

```

Common Mistakes

1. **Storing encryption keys in the database** — the key and locked data are in the same place
 2. **Using MD5 for password hashing** — use bcrypt/scrypt instead
 3. **ECB mode (aes-ecb)** — identical plaintexts produce identical ciphertexts (pattern leak)
 4. **Not using gen_salt('bf') with bcrypt** — reusing salts destroys security
 5. **Encrypting data you need to search** — use hashing/tokenization for searchable fields
 6. **Hardcoding keys in SQL functions** — keys visible in pg_proc
 7. **Not rotating keys periodically** — key compromise has long-term impact
 8. **Encrypting too little** — missing sensitive columns
 9. **Encrypting too much** — performance penalty on non-sensitive data
 10. **No key backup** — encrypted data becomes unrecoverable if key is lost
-

Best Practices

1. **Never store encryption keys in the database** — use environment variables or Vault
 2. **Use pgp_sym_encrypt (bcrypt-derived key) over raw_encrypt()** for column encryption
 3. **Use bcrypt with work factor 10-12** for password hashing
 4. **Index hash columns** for searchable encrypted fields (tokenization pattern)
 5. **Separate key management** from data management
 6. **Rotate keys periodically** — implement re-encryption procedures
 7. **Combine TDE + column encryption** for defense in depth
 8. **Audit decryption operations** — log when encrypted data is accessed
 9. **Test key recovery** — ensure you can decrypt after key rotation
 10. **Use the SECURITY DEFINER pattern** to hide encryption keys from app users
-

Interview Questions

Q1: What is the difference between encryption and hashing?

A: Encryption is reversible (decrypt with the right key). Hashing is one-way — you cannot recover the original value. Use encryption when you need to retrieve the original value (SSN, credit card). Use hashing when you only need to verify the value (passwords — store the hash, verify by hashing the attempt and comparing).

Q2: Why shouldn't encryption keys be stored in the database?

A: If an attacker gains database access, they get both the encrypted data and the key — rendering encryption useless. Keys should be stored externally (HashiCorp Vault, AWS KMS, environment variables) so that accessing the database alone is insufficient to decrypt data.

Q3: What is pgcrypto and what does it provide?

A: pgcrypto is a PostgreSQL extension providing cryptographic functions: (1) Symmetric encryption: `encrypt()` / `decrypt()` using AES, 3DES, Blowfish. (2) PGP encryption: `pgp_sym_encrypt` / `pgp_pub_encrypt` using public key cryptography. (3) Password hashing: `crypt()` with bcrypt/SHA-256/MD5. (4) General hashing: `digest()` for SHA-256/SHA-512/MD5. (5) HMAC: `hmac()` for keyed authentication. (6) Random generation: `gen_random_bytes()` , `gen_random_uuid()` .

Q4: How do you search encrypted data?

A: You generally cannot search ciphertext directly. Solutions: (1) Tokenization — store a hash of the sensitive value as an index-able search token alongside the encrypted value. (2) Deterministic encryption — use the same key+IV for the same plaintext (allows equality search, but leaks which rows have identical values). (3) Searchable encryption schemes (academic, complex). Most practical: store encrypted value + hash token + last-N-chars for display.

Q5: What is the difference between bcrypt and SHA-256 for password storage?

A: SHA-256 is fast (designed for performance) — attackers can compute billions of hashes per second for brute-force. bcrypt is intentionally slow (configurable work factor): each hash takes ~100ms at work factor 10. This makes brute-force attacks 10-million times slower. bcrypt also includes a salt automatically, preventing rainbow tables. Always use bcrypt (or scrypt/Argon2) for passwords, never raw SHA/MD5.

Q6: What is Transparent Data Encryption (TDE)?

A: TDE encrypts data files at the storage layer, automatically decrypting on read and encrypting on write. Applications and queries don't change. It protects against physical theft of storage media but NOT against authorized database connections. PostgreSQL lacks native TDE (open source) — it's typically achieved via OS filesystem encryption (LUKS), cloud storage encryption (EBS), or commercial distributions (EDB, Percona).

Q7: What is envelope encryption?

A: A two-tier key scheme: Data is encrypted with a random Data Encryption Key (DEK). The DEK itself is encrypted with a Key Encryption Key (KEK) managed externally (Vault/KMS). Only the encrypted DEK is stored with the data. Advantages: (1) The expensive KMS call only happens when the DEK needs to change, not for every operation. (2) Key rotation only requires re-encrypting the DEK, not all data.

Q8: How do you handle key rotation for existing encrypted data?

A: (1) Generate new key. (2) For each row: decrypt with old key, re-encrypt with new key. (3) Atomic swap if needed (write to new column, verify, drop old). For large tables: batch process, background job, or use logical replication to the new schema. The critical requirement is being able to decrypt with the old key and the new key during the transition.

Exercises and Solutions

Exercise 1: Implement Encrypted PII Storage

```

-- Encrypt and safely store PII (SSN, DOB, Credit Card)
-- Application passes encryption key via session var

CREATE TABLE customer_pii (
    id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES customers(id),
    -- Encrypted PII
    ssn_encrypted BYTEA,
    dob_encrypted BYTEA,
    credit_card_encrypted BYTEA,
    -- Searchable tokens
    ssn_last_four CHAR(4),      -- Last 4 of SSN for display
    card_last_four CHAR(4),    -- Last 4 of card for display
    card_bin CHAR(6),          -- First 6 for BIN lookup
    -- Integrity
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- Store encrypted PII
CREATE OR REPLACE FUNCTION store_pii(
    p_customer_id INT,
    p_ssn TEXT,
    p_dob DATE,
    p_credit_card TEXT
) RETURNS INT AS $$
DECLARE
    enc_key TEXT := current_setting('app.encrypted_key');
    new_id INT;
BEGIN
    INSERT INTO customer_pii (
        customer_id, ssn_encrypted, dob_encrypted,
        credit_card_encrypted, ssn_last_four, card_last_four, card_bin
    ) VALUES (
        p_customer_id,
        pgp_sym_encrypt(p_ssn, enc_key),
        pgp_sym_encrypt(p_dob::text, enc_key),
        pgp_sym_encrypt(p_credit_card, enc_key),
        right(p_ssn, 4),
        right(p_credit_card, 4),
        left(p_credit_card, 6)
    )
    RETURNING id INTO new_id;

    RETURN new_id;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Usage:

```

```
SET LOCAL app.encrypted_key = 'my_secure_32_byte_encryption_key';  
SELECT store_pii(1, '123-45-6789', '1990-01-15', '4111111111111111');
```

Cross-References

- [04 ssl tls.md](#) — Encryption in transit
- [07 secrets management.md](#) — Key management with Vault
- [08 compliance considerations.md](#) — GDPR data encryption requirements
- [../15 Backup Recovery/06 pgbackrest.md](#) — Backup encryption